```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>KASE ERP – Placements</title>
<style>
:root{--navy:#004578;--blue:#0067a8;--pale:#eef5fa;--line:#d6dde3;--ink:#202a33;--muted:#5d6974;--bg:#f4f6f8;--white:#fff;--green:#18794e;--amber:#946200;--red:#b42318}
*{box-sizing:border-box}
body{margin:0;background:var(--bg);color:var(--ink);font:14px/1.45 Arial,Helvetica,sans-serif}
button,input,select,textarea{font:inherit}
button{cursor:pointer}
header{height:64px;background:var(--navy);color:#fff;display:flex;align-items:center;justify-content:space-between;padding:0 22px;gap:16px}
header .brand{display:flex;align-items:center;gap:12px;min-width:0}
header .brand strong{font-size:17px}
header .brand span{color:#c6d8e7}
header .right{display:flex;align-items:center;gap:10px}
header input{border:1px solid #9ab0c1;border-radius:4px;padding:9px 10px;width:240px}
header button{background:#fff;color:var(--navy);border:0;border-radius:4px;padding:8px 12px}
.layout{display:grid;grid-template-columns:245px minmax(0,1fr);min-height:calc(100vh - 108px)}
aside{background:#fff;border-right:1px solid var(--line);padding:16px 0}
.nav-section{padding:13px 18px 6px;color:#5d6974;font-size:11px;text-transform:uppercase;letter-spacing:.06em;font-weight:700}
.nav-item{display:block;width:100%;border:0;border-left:3px solid transparent;background:#fff;text-align:left;padding:11px 20px;color:#243746}
.nav-item:hover{background:#f4f7f9}
.nav-item.active{background:#e8eef3;border-left-color:var(--navy);font-weight:700;color:var(--navy)}
main{padding:22px;min-width:0}
```

```
.page-head{display:flex;align-items:flex-start;justify-content:space-between;gap:15px;margin-bottom:18px}
.page-head h1{font-size:24px;margin:0 0 5px;font-weight:700}
.crumb{color:var(--muted);font-size:12px}
.actions{display:flex;flex-wrap:wrap;gap:8px}
.btn{border:1px solid #bdc7cf;background:#fff;color:#243746;padding:8px 12px;border-radius:4px}
.btn:hover{background:#f4f7f9}
.btn.primary{background:var(--navy);border-color:var(--navy);color:#fff}
.btn.primary:hover{background:#00375e}
.btn.danger{border-color:#d9aaa6;color:var(--red)}
.btn.small{padding:5px 8px;font-size:12px}
.cards{display:grid;grid-template-columns:repeat(4,minmax(0,1fr));gap:12px;margin-bottom:16px}
.metric,.card{background:#fff;border:1px solid var(--line);border-radius:5px}
.metric{padding:16px}
.metric .label{color:var(--muted);font-size:12px}
.metric .value{font-size:27px;font-weight:700;margin:6px 0 2px}
.metric .hint{font-size:12px;color:var(--muted)}
.card{padding:16px;margin-bottom:16px;min-width:0}
.card-head{display:flex;align-items:center;justify-content:space-between;gap:12px;margin-bottom:12px}
.card h2,.card h3{font-size:16px;margin:0}
.card p{margin:5px 0;color:var(--muted)}
.filters{display:flex;flex-wrap:wrap;gap:8px;margin-bottom:12px}
.filters input,.filters select{min-width:150px;max-width:100%;padding:8px;border:1px solid #c8d0d7;border-radius:4px;background:#fff}
.table-wrap{overflow:auto;border:1px solid var(--line);border-radius:4px}
table{width:100%;border-collapse:collapse;background:#fff;white-space:nowrap}
th,td{text-align:left;padding:10px 11px;border-bottom:1px solid #e3e7eb;vertical-align:middle}
th{background:#f2f5f7;font-size:12px;color:#394957;font-weight:700}
tbody tr:hover{background:#fafcfd}
tbody tr:last-child td{border-bottom:0}
```

.link-btn{background:none;border:0;color:#005b91;padding:0;text-decoration:underline;text-underline-offset:2px}

.muted{color:var(--muted)}

.status{display:inline-block;padding:3px 7px;border-radius:3px;background:#edf0f2;color:#3c4852;font-size:11px;font-weight:700}

.status.good{background:#e5f4eb;color:var(--green)}

.status.warn{background:#fff2d2;color:var(--amber)}

.status.bad{background:#feeceb;color:var(--red)}

.empty{padding:24px;text-align:center;color:var(--muted)}

.grid-two{display:grid;grid-template-columns:repeat(2,minmax(0,1fr));gap:14px}

.field{display:flex;flex-direction:column;gap:5px;margin-bottom:12px}

.field label{font-size:12px;font-weight:700;color:#354553}

.field input,.field select,.field textarea{width:100%;border:1px solid #c8d0d7;border-radius:4px;padding:9px;background:#fff}

.field textarea{min-height:85px;resize:vertical}

.field .help{font-size:11px;color:var(--muted)}

.form-grid{display:grid;grid-template-columns:repeat(2,minmax(0,1fr));gap:0 12px}

.form-grid .full{grid-column:1/-1}

.modal-overlay{position:fixed;inset:0;background:#17232f66;display:none;align-items:center;justify-content:center;padding:18px;z-index:50}

.modal-overlay.show{display:flex}

.modal{background:#fff;border-radius:6px;width:min(760px,100%);max-height:90vh;display:flex;flex-direction:column;box-shadow:0 15px 45px #0002}

.modal-header{padding:16px 18px;border-bottom:1px solid var(--line);display:flex;justify-content:space-between;align-items:center;gap:12px}

.modal-header h2{font-size:18px;margin:0}

.modal-header button{border:0;background:transparent;font-size:14px;color:#4b5964}

.modal-body{padding:18px;overflow:auto}

.modal-footer{display:flex;justify-content:flex-end;gap:8px;padding:14px 18px;border-top:1px solid var(--line)}

.detail-list{display:grid;grid-template-columns:repeat(2,minmax(0,1fr));gap:10px}

.detail{padding:10px;border:1px solid #e1e6ea;border-radius:4px;min-width:0}

.detail .k{font-size:11px;color:var(--muted);margin-bottom:4px}

```
.detail .v{font-weight:600;overflow-wrap:anywhere}

.toast{position:fixed;right:20px;bottom:20px;background:#253746;color:#fff;padding:11px 15px;border-radius:4px;display:none;z-index:100;max-width:min(440px,calc(100vw - 40px))}

.toast.show{display:block}

.login-wrap{position:fixed;inset:0;display:flex;align-items:center;justify-content:center;background:#edf2f6;z-index:200;padding:20px}

.login-card{width:min(430px,100%);background:#fff;border:1px solid var(--line);border-radius:6px;padding:26px;box-shadow:0 8px 30px #0001}

.login-card h1{font-size:22px;margin:0 0 5px;color:var(--navy)}

.login-card p{color:var(--muted);margin:0 0 18px}

.login-card .btn{width:100%;margin-top:5px}

.login-help{font-size:12px;background:#f3f6f8;padding:10px;border-radius:4px;margin-top:15px;line-height:1.6}

.note{border-left:3px solid #7d9bb1;padding:10px 12px;background:#f3f7fa;color:#3e4d58;margin:10px 0}

.checkline{display:flex;gap:8px;align-items:center;font-size:13px;margin:8px 0}

.checkline input{width:auto}

footer{height:44px;display:flex;align-items:center;justify-content:space-between;padding:0 22px;border-top:1px solid var(--line);background:#fff;color:var(--muted);font-size:11px}

@media(max-width:1050px){

 .cards{grid-template-columns:repeat(2,minmax(0,1fr))}

 .layout{grid-template-columns:205px minmax(0,1fr)}

 main{padding:16px}

}

@media(max-width:720px){

 header{height:auto;min-height:64px;align-items:flex-start;padding:12px;flex-direction:column}

 header .right{width:100%;flex-wrap:wrap}

 header input{flex:1;width:100%;min-width:130px}

 .layout{display:block}

 aside{display:flex;overflow:auto;padding:0;border-right:0;border-bottom:1px solid var(--line)}

 .nav-section{display:none}

 .nav-item{width:auto;white-space:nowrap;border-left:0;border-bottom:3px solid transparent;padding:12px}
```

```
 .nav-item.active{border-left:0;border-bottom-color:var(--navy)}
 main{padding:12px}
 .page-head{flex-direction:column}
 .cards{grid-template-columns:repeat(2,minmax(0,1fr))}
 .grid-two,.form-grid,.detail-list{grid-template-columns:1fr}
 .form-grid .full{grid-column:auto}
 footer{padding:8px 12px;height:auto;gap:8px;flex-wrap:wrap}
}
</style>
</head>
<body>
<header>
  <div class="brand"><strong>KASE
ERP</strong><span>|</span><strong>Placements</strong></div>
  <div class="right">
    <input id="globalSearch" placeholder="Search placement records">
    <span id="userLabel">Not signed in</span>
    <button id="logoutButton" onclick="logout()">Sign out</button>
  </div>
</header>
<div class="layout">
  <aside id="sidebar"></aside>
  <main id="main"></main>
</div>
<footer>
  <span>Kerala Academy for Skills Excellence · Placements Module</span>
  <span id="dateLabel"></span>
</footer>
<div class="modal-overlay" id="modalOverlay" onclick="overlayClose(event)">
  <div class="modal" role="dialog" aria-modal="true">
    <div class="modal-header">
```

```
    <h2 id="modalTitle">Details</h2>
    <button aria-label="Close" onclick="closeModal()">Close</button>
   </div>
   <div class="modal-body" id="modalBody"></div>
   <div class="modal-footer" id="modalFooter"></div>
  </div>
</div>
<div class="toast" id="toast"></div>
<div class="login-wrap" id="loginWrap">
  <form class="login-card" onsubmit="login(event)">
   <h1>KASE ERP – Placements</h1>
   <p>Internal placement management</p>
   <div class="field">
    <label for="loginId">User ID</label>
    <input id="loginId" required autocomplete="username" placeholder="manager or
executive.placements">
   </div>
   <div class="field">
    <label for="loginPassword">Password</label>
    <input id="loginPassword" required type="password" autocomplete="current-password"
placeholder="Enter password">
   </div>
   <button class="btn primary" type="submit">Sign in</button>
   <div class="login-help">
    Demo accounts:<br>
    Manager (Placements): <b>manager</b><br>
    Executive (Placements): <b>executive.placements</b><br>
    Password for both: <b>Demo@123</b>
   </div>
  </form>
</div>
<script>
```

```
"use strict";

const $ = id => document.getElementById(id);
const isoToday = () => new Date().toISOString().slice(0,10);
const fmtDate = value => value ? new Date(value + (String(value).length===10 ? "T00:00:00" : "")).toLocaleDateString("en-GB") : "—";
const daysAfter = (date, days) => {
 if(!date) return "";
 const d = new Date(date+"T00:00:00");
 d.setDate(d.getDate()+days);
 return d.toISOString().slice(0,10);
};
const money = n => "Rs. " + Number(n || 0).toLocaleString("en-IN", {maximumFractionDigits:2});
const esc = s => String(s ?? "").replace(/[&<>"']/g, c => ({"&":"&amp;","<":"&lt;",">":"&gt;",'"':"&quot;","'":"&#39;"}[c]));
const id = prefix => prefix + "-" + Date.now().toString().slice(-7);

const accounts = {
 manager: { password: "Demo@123", role: "manager", label: "Manager (Placements)" },
 "executive.placements": { password: "Demo@123", role: "executive", label: "Executive (Placements)" }
};

const navItems = [
 {
  section:"Placement Management",
  items:[
   ["dashboard","Dashboard"],
   ["talent","Talent Pool"],
   ["employers","Employer Register"],
   ["vacancies","Vacancies & Matching"],
   ["drives","Placement Drives"],
```

```
      ["interviews","Interview Pipeline"]
    ]
  },
  {
    section:"Verification & Payments",
    items:[
      ["verification","Employment Verification"],
      ["retention","Retention Milestones"],
      ["fico","FICO & eOffice"],
      ["reports","Reports & Audit"]
    ]
  }
];

const seed = {
  candidates:[
    {
      id:"CAN-26001",name:"Anjana S",district:"Thiruvananthapuram",
      qualification:"Diploma",skill:"Electric Vehicle Service",
      scheme:"State Skill Programme",batch:"BATCH-104",certificate:"CERT-26001",
      certified:true,phone:"98XXXX2101",email:"anjana.demo@example.com",
      visibility:"Visible to verified employers",resume:"Anjana_S_Resume.pdf",
      status:"Available",employerId:"",jobId:"",placedAt:"",salary:0,
      proof:"",proofStatus:"Not Submitted",sanctionStatus:"Pending",
      retention:{},history:[]
    },
    {
      id:"CAN-26002",name:"Rahul P",district:"Kozhikode",
      qualification:"ITI",skill:"Industrial Automation",scheme:"PMKVY",
      batch:"BATCH-103",certificate:"CERT-26002",certified:true,
      phone:"97XXXX3202",email:"rahul.demo@example.com",
```

```
 visibility:"Visible to verified employers",resume:"Rahul_P_CV.pdf",
 status:"Available",employerId:"",jobId:"",placedAt:"",salary:0,
 proof:"",proofStatus:"Not Submitted",sanctionStatus:"Pending",
 retention:{},history:[]
},
{
 id:"CAN-26003",name:"Meera K",district:"Ernakulam",
 qualification:"Degree",skill:"Digital Marketing",
 scheme:"State Skill Programme",batch:"BATCH-101",
 certificate:"CERT-26003",certified:true,phone:"96XXXX4303",
 email:"meera.demo@example.com",
 visibility:"Visible to verified employers",resume:"Meera_K_Resume.pdf",
 status:"Interviewing",employerId:"EMP-1002",jobId:"JOB-2002",
 placedAt:"",salary:0,proof:"",proofStatus:"Not Submitted",
 sanctionStatus:"Pending",retention:{},history:[]
},
{
 id:"CAN-26004",name:"Nikhil R",district:"Thrissur",
 qualification:"ITI",skill:"Electrical Maintenance",scheme:"PMKVY",
 batch:"BATCH-102",certificate:"CERT-26004",certified:true,
 phone:"95XXXX5404",email:"nikhil.demo@example.com",
 visibility:"Visible to verified employers",resume:"Nikhil_R_CV.pdf",
 status:"Placed",employerId:"EMP-1001",jobId:"JOB-2001",
 placedAt:daysAfter(isoToday(),-46),salary:18500,
 proof:"Appointment_Nikhil.pdf",proofStatus:"Verified",
 sanctionStatus:"Sanctioned",
 retention:{
  30:{status:"Verified",proof:"Salary slip month 1.pdf",date:daysAfter(isoToday(),-15)},
  60:{status:"Pending",proof:"",date:""},
  90:{status:"Pending",proof:"",date:""}
 },
```

```
    history:[]
  },
  {
    id:"CAN-26005",name:"Fathima M",district:"Malappuram",
    qualification:"Plus Two",skill:"Customer Relationship Management",
    scheme:"State Skill Programme",batch:"BATCH-105",
    certificate:"CERT-26005",certified:true,phone:"94XXXX6505",
    email:"fathima.demo@example.com",
    visibility:"Visible to verified employers",resume:"Fathima_M_CV.pdf",
    status:"Available",employerId:"",jobId:"",placedAt:"",salary:0,
    proof:"",proofStatus:"Not Submitted",sanctionStatus:"Pending",
    retention:{},history:[]
  }
],
employers:[
  {
    id:"EMP-1001",name:"Malabar Engineering Works",
    sector:"Engineering",district:"Thrissur",contact:"Placement Desk",
    email:"hr@example.com",phone:"0484-0000000",status:"Verified",
    verifiedBy:"Manager (Placements)",verifiedAt:isoToday(),
    notes:"Demo employer record"
  },
  {
    id:"EMP-1002",name:"Kerala Digital Services",
    sector:"IT & Digital",district:"Ernakulam",contact:"HR Team",
    email:"careers@example.com",phone:"0484-1111111",status:"Verified",
    verifiedBy:"Manager (Placements)",verifiedAt:isoToday(),
    notes:"Demo employer record"
  },
  {
    id:"EMP-1003",name:"Coastal Automation LLP",
```

```
    sector:"Manufacturing",district:"Alappuzha",
    contact:"Recruitment Officer",email:"recruit@example.com",
    phone:"0477-2222222",status:"Pending Verification",
    verifiedBy:"",verifiedAt:"",notes:"Awaiting credential review"
  }
 ],
 jobs:[
  {
    id:"JOB-2001",employerId:"EMP-1001",
    title:"Electrical Maintenance Technician",district:"Thrissur",
    skill:"Electrical Maintenance",qualification:"ITI",positions:4,
    salary:18500,workMode:"On-site",postedAt:isoToday(),
    deadline:daysAfter(isoToday(),30),status:"Open",
    description:"Electrical maintenance and routine service work."
  },
  {
    id:"JOB-2002",employerId:"EMP-1002",
    title:"Digital Marketing Associate",district:"Ernakulam",
    skill:"Digital Marketing",qualification:"Degree",positions:3,
    salary:22000,workMode:"Hybrid",postedAt:isoToday(),
    deadline:daysAfter(isoToday(),20),status:"Open",
    description:"Campaign support, reporting and content coordination."
  },
  {
    id:"JOB-2003",employerId:"EMP-1003",
    title:"Automation Trainee",district:"Alappuzha",
    skill:"Industrial Automation",qualification:"ITI",positions:2,
    salary:20000,workMode:"On-site",postedAt:isoToday(),
    deadline:daysAfter(isoToday(),25),status:"Draft",
    description:"Assist with industrial automation maintenance."
  }
```

```
],
drives:[
 {
  id:"DRV-3001",title:"Electrical Maintenance Recruitment Drive",
  employerId:"EMP-1001",jobId:"JOB-2001",
  date:daysAfter(isoToday(),8),venue:"KASE / Online",mode:"Hybrid",
  status:"Scheduled",candidateIds:["CAN-26001","CAN-26004"],
  attendance:{"CAN-26004":"Present"},createdAt:isoToday()
 },
 {
  id:"DRV-3002",title:"Digital Services Interview Day",
  employerId:"EMP-1002",jobId:"JOB-2002",
  date:daysAfter(isoToday(),14),venue:"Online",mode:"Online",
  status:"Scheduled",candidateIds:["CAN-26003"],attendance:{},
  createdAt:isoToday()
 }
],
interviews:[
 {
  id:"INT-4001",candidateId:"CAN-26003",employerId:"EMP-1002",
  jobId:"JOB-2002",driveId:"DRV-3002",stage:"Interview Scheduled",
  date:daysAfter(isoToday(),14),employerUpdate:"Shortlisted for interview",
  updatedAt:isoToday(),notes:"Demo pipeline record"
 },
 {
  id:"INT-4002",candidateId:"CAN-26001",employerId:"EMP-1001",
  jobId:"JOB-2001",driveId:"DRV-3001",stage:"Invited",
  date:daysAfter(isoToday(),8),employerUpdate:"Invitation created",
  updatedAt:isoToday(),notes:"Demo pipeline record"
 }
],
```

```
  fico:[
    {
      id:"FICO-5001",candidateId:"CAN-26004",
      milestone:"30-day retention",amount:5000,
      ledger:"Placements Support",eoffice:"EFILE/PLC/2026/001",
      report:"FICO_30DAY_CAN-26004.pdf",
      status:"Submitted for AP Processing",createdAt:isoToday(),history:[]
    }
  ],
  audit:[]
};

let db = loadDb();
let role = null;
let activeModule = "dashboard";
let currentToast = null;
let loggedInId = "";

function loadDb(){
  try{
    const saved=localStorage.getItem("kasePlacementsInternalV1");
    if(saved){
      const parsed=JSON.parse(saved);
      return {...structuredClone(seed),...parsed};
    }
  }catch(e){
    console.warn("Could not load saved demo data",e);
  }
  return structuredClone(seed);
}
```

```
function saveDb(){
 localStorage.setItem("kasePlacementsInternalV1",JSON.stringify(db));
}

function logAction(action,entity,details=""){
 db.audit.unshift({
  id:id("AUD"),
  date:new Date().toLocaleString("en-GB"),
  user:accounts[loggedInId]?.label||"Internal user",
  action,entity,details
 });
 saveDb();
}

function login(event){
 event.preventDefault();
 const username=$("loginId").value.trim();
 const password=$("loginPassword").value;
 const account=accounts[username];

 if(!account||account.password!==password){
  toast("Invalid user ID or password");
  return;
 }

 role=account.role;
 loggedInId=username;
 $("loginWrap").style.display="none";
 $("userLabel").textContent=account.label;
 $("logoutButton").style.display="inline-block";
 logAction("Sign in","Session",account.label);
```

```
  render();
}

function logout(){
  if(role) logAction("Sign out","Session",accounts[loggedInId]?.label||"");
  role=null;
  loggedInId="";
  $("loginPassword").value="";
  $("userLabel").textContent="Not signed in";
  $("loginWrap").style.display="flex";
}

function canManage(){
  return role==="manager";
}

function toast(message){
  const el=$("toast");
  el.textContent=message;
  el.classList.add("show");
  clearTimeout(currentToast);
  currentToast=setTimeout(()=>el.classList.remove("show"),3200);
}

function openModule(module){
  activeModule=module;
  render();
}

function renderSidebar(){
  let html="";
```

```
  for(const section of navItems){
    html+=`<div class="nav-section">${esc(section.section)}</div>`;
    for(const [key,label] of section.items){
      html+=`<button class="nav-item ${activeModule===key?"active":""}" onclick="openModule('${key}')">${esc(label)}</button>`;
    }
  }
  $("sidebar").innerHTML=html;
}

function pageHead(title,crumb,actions=""){
  return `<div class="page-head"><div><h1>${esc(title)}</h1><div class="crumb">${esc(crumb)}</div></div><div class="actions">${actions}</div></div>`;
}

function metric(label,value,hint=""){
  return `<div class="metric"><div class="label">${esc(label)}</div><div class="value">${esc(value)}</div><div class="hint">${esc(hint)}</div></div>`;
}

function statusBadge(value){
  const v=String(value||"Pending");
  let cls="";
  if(/verified|sanctioned|present|placed|approved|complete|open/i.test(v)) cls="good";
  else if(/pending|scheduled|invited|review|submitted|interview/i.test(v)) cls="warn";
  else if(/rejected|expired|not selected|closed|failed/i.test(v)) cls="bad";
  return `<span class="status ${cls}">${esc(v)}</span>`;
}

function table(headers,rows,cellFn,empty="No records found."){
  if(!rows.length) return `<div class="empty">${esc(empty)}</div>`;
```

```
  return `<div class="table-
wrap"><table><thead><tr>${headers.map(h=>`<th>${esc(h)}</th>`).join("")}</tr></thead><tbody>$
{rows.map(row=>`<tr>${cellFn(row).map(v=>`<td>${v==null?"—
":v}</td>`).join("")}</tr>`).join("")}</tbody></table></div>`;
}

function btn(label,action,kind=""){
  return `<button class="btn ${kind}" onclick="${action}">${esc(label)}</button>`;
}

function getCandidate(cid){return db.candidates.find(c=>c.id===cid);}
function getEmployer(eid){return db.employers.find(e=>e.id===eid);}
function getJob(jid){return db.jobs.find(j=>j.id===jid);}
function getDrive(did){return db.drives.find(d=>d.id===did);}
function candidateName(cid){return getCandidate(cid)?.name||cid||"—";}
function employerName(eid){return getEmployer(eid)?.name||eid||"—";}
function jobTitle(jid){return getJob(jid)?.title||jid||"—";}

function setModal(title,body,buttons=[]){
  $("modalTitle").textContent=title;
  $("modalBody").innerHTML=body;
  $("modalFooter").innerHTML=buttons.map(b=>`<button class="btn ${b.kind||""}"
onclick="${b.action}">${esc(b.label)}</button>`).join("");
  $("modalOverlay").classList.add("show");
}

function closeModal(){
  $("modalOverlay").classList.remove("show");
  $("modalBody").innerHTML="";
}

function overlayClose(event){
```

```
  if(event.target===$("modalOverlay")) closeModal();
}

function detailItem(k,v){
  return `<div class="detail"><div class="k">${esc(k)}</div><div class="v">${v==null||v===""?"—
":esc(v)}</div></div>`;
}

function recordDetails(title,record,extra=[]){
  const all=[
    ...Object.entries(record).filter(([k,v])=>!Array.isArray(v)&&typeof
v!=="object"&&!/password/i.test(k)),
    ...extra
  ];
  return `<div class="detail-list">${all.map(([k,v])=>detailItem(k.replace(/([A-Z])/g,"
$1").replace(/^./,s=>s.toUpperCase()),v)).join("")}</div>`;
}

function candidateOptions(value=""){
  return db.candidates.map(c=>`<option value="${esc(c.id)}"
${c.id===value?"selected":""}>${esc(c.id+" – "+c.name)}</option>`).join("");
}

function employerOptions(value=""){
  return db.employers.map(e=>`<option value="${esc(e.id)}"
${e.id===value?"selected":""}>${esc(e.id+" – "+e.name)}</option>`).join("");
}

function jobOptions(value=""){
  return db.jobs.map(j=>`<option value="${esc(j.id)}" ${j.id===value?"selected":""}>${esc(j.id+" –
"+j.title)}</option>`).join("");
}
```

```
function driveOptions(value=""){
 return db.drives.map(d=>`<option value="${esc(d.id)}" ${d.id===value?"selected":""}>${esc(d.id+" – "+d.title)}</option>`).join("");
}

function render(){
 renderSidebar();
 if(!role){
  $("main").innerHTML="";
  return;
 }
 const routes={
  dashboard:renderDashboard,
  talent:renderTalent,
  employers:renderEmployers,
  vacancies:renderVacancies,
  drives:renderDrives,
  interviews:renderInterviews,
  verification:renderVerification,
  retention:renderRetention,
  fico:renderFico,
  reports:renderReports
 };
 $("main").innerHTML=(routes[activeModule]||renderDashboard)();
}

function renderDashboard(){
 const certified=db.candidates.filter(c=>c.certified).length;
 const placed=db.candidates.filter(c=>c.status==="Placed"||c.status==="Retention Monitoring").length;
 const pendingProof=db.candidates.filter(c=>c.status==="Awaiting Verification"||c.proofStatus==="Submitted").length;
```

```
  const pendingEmployers=db.employers.filter(e=>e.status!=="Verified").length;


  return pageHead("Placements Dashboard","KASE ERP › Placements",btn("Export summary","exportDashboard()"))+
  `<div class="cards">
    ${metric("Certified candidates",certified,"Eligible for talent pool")}
    ${metric("Placed candidates",placed,"Including retention monitoring")}
    ${metric("Employment proof pending",pendingProof,"Verification required")}
    ${metric("Employers to verify",pendingEmployers,"Employer register")}
  </div>
  <div class="grid-two">
    <section class="card">
      <div class="card-head"><h2>Placement Activity</h2>${btn("Open talent pool","openModule('talent')")}</div>
      ${table(["Candidate","Skill","Status","Employer"],db.candidates.slice(0,5),c=>[
        `<button class="link-btn" onclick="viewCandidate('${c.id}')">${esc(c.name)}</button>`,
        esc(c.skill),statusBadge(c.status),esc(employerName(c.employerId))
      ])}
    </section>
    <section class="card">
      <div class="card-head"><h2>Pending Actions</h2></div>
      ${table(["Record","Action Required","Status"],[
        ...db.employers.filter(e=>e.status!=="Verified").map(e=>({record:e.name,action:"Employer verification",status:e.status,fn:`viewEmployer('${e.id}')`})),
        ...db.candidates.filter(c=>c.proofStatus==="Submitted"||c.status==="Awaiting Verification").map(c=>({record:c.name,action:"Employment proof review",status:c.proofStatus,fn:`viewVerification('${c.id}')`}))
      ].slice(0,7),r=>[
        `<button class="link-btn" onclick="${r.fn}">${esc(r.record)}</button>`,
        esc(r.action),statusBadge(r.status)
      ],"No pending actions.")}
    </section>
  </div>
```

```
 <section class="card">
  <div class="card-head"><h2>Upcoming Placement Drives</h2>${btn("View all drives","openModule('drives')")}</div>

${table(["Drive","Employer","Date","Mode","Candidates","Status"],db.drives.filter(d=>d.status!=="Completed").sort((a,b)=>a.date.localeCompare(b.date)),d=>[
    `<button class="link-btn" onclick="viewDrive('${d.id}')">${esc(d.title)}</button>`,

esc(employerName(d.employerId)),fmtDate(d.date),esc(d.mode),d.candidateIds.length,statusBadge(d.status)
  ])}
 </section>`;
}

function renderTalent(){
 const q=($("globalSearch").value||"").trim().toLowerCase();
 const rows=db.candidates.filter(c=>c.certified&&(!q||JSON.stringify(c).toLowerCase().includes(q)));
 return pageHead("Talent Pool","KASE ERP › Placements › Talent Pool",btn("Add candidate record","candidateForm()","primary")+btn("Export CSV","exportRows('candidates')"))+
 `<div class="note">Only candidates with recorded certification are included in the talent pool. Candidate registrations and external profile updates are expected to come through the KASE website integration; this page is for internal management.</div>
 <div class="filters">
  <input placeholder="Search candidate, skill or district" value="${esc(q)}" oninput="filterTalent(this.value)">
  <select onchange="filterTalentStatus(this.value)" id="talentStatusFilter">
   <option value="">All placement statuses</option>
   ${["Available","Shortlisted","Interviewing","Placed","Retention Monitoring","Awaiting Verification","Not Selected"].map(s=>`<option>${s}</option>`).join("")}
  </select>
  <select id="talentVisibilityFilter" onchange="filterTalentVisibility(this.value)">
   <option value="">All visibility settings</option>
   <option>Visible to verified employers</option>
   <option>Hidden</option>
```

```
  </select>
 </div>
 <div id="talentTable">${talentTable(rows)}</div>`;
}

function talentTable(rows){
 return table(["Candidate ID","Candidate","District","Skill","Certificate","Visibility","Status","Actions"],rows,c=>[
  esc(c.id),
  `<button class="link-btn" onclick="viewCandidate('${c.id}')">${esc(c.name)}</button>`,
  esc(c.district),esc(c.skill),esc(c.certificate),esc(c.visibility),statusBadge(c.status),
  btn("View",`viewCandidate('${c.id}')`,"small")
 ],"No certified candidates match the search.");
}

function filterTalent(value){
 const q=value.toLowerCase();
 const status=$("talentStatusFilter")?.value||"";
 const vis=$("talentVisibilityFilter")?.value||"";
 const rows=db.candidates.filter(c=>c.certified&&JSON.stringify(c).toLowerCase().includes(q)&&(!status||c.status===status)&&(!vis||c.visibility===vis));
 $("talentTable").innerHTML=talentTable(rows);
}
function filterTalentStatus(value){const search=document.querySelector(".filters input")?.value||"";filterTalent(search);}
function filterTalentVisibility(value){const search=document.querySelector(".filters input")?.value||"";filterTalent(search);}

function candidateForm(cid=""){
 const c=cid?getCandidate(cid):{
  id:"",name:"",district:"",qualification:"",skill:"",scheme:"",batch:"",
  certificate:"",certified:true,phone:"",email:"",
```

```
   visibility:"Visible to verified employers",resume:"",status:"Available"
  };
  const isNew=!cid;
  const body=`<form id="candidateForm" onsubmit="saveCandidate(event,'${esc(cid)}')">
    <div class="form-grid">
     <div class="field"><label>Candidate ID</label><input name="id" value="${esc(c.id||"")}"
${isNew?"":"readonly"} required></div>
     <div class="field"><label>Full name</label><input name="name" value="${esc(c.name)}"
required></div>
     <div class="field"><label>District</label><input name="district" value="${esc(c.district)}"
required></div>
     <div class="field"><label>Qualification</label><input name="qualification"
value="${esc(c.qualification)}"></div>
     <div class="field"><label>Skill / Trade</label><input name="skill" value="${esc(c.skill)}"
required></div>
     <div class="field"><label>Scheme</label><input name="scheme"
value="${esc(c.scheme)}"></div>
     <div class="field"><label>Batch reference</label><input name="batch"
value="${esc(c.batch)}"></div>
     <div class="field"><label>Certificate reference</label><input name="certificate"
value="${esc(c.certificate)}" required></div>
     <div class="field"><label>Contact number (mask as needed)</label><input name="phone"
value="${esc(c.phone)}"></div>
     <div class="field"><label>Email</label><input name="email" type="email"
value="${esc(c.email)}"></div>
     <div class="field"><label>Talent-pool visibility</label><select name="visibility">
       <option ${c.visibility==="Visible to verified employers"?"selected":""}>Visible to verified
employers</option>
       <option ${c.visibility==="Hidden"?"selected":""}>Hidden</option>
     </select></div>
     <div class="field"><label>Placement status</label><select name="status">
       ${["Available","Shortlisted","Interviewing","Placed","Retention Monitoring","Awaiting
Verification","Not Selected"].map(s=>`<option
${c.status===s?"selected":""}>${s}</option>`).join("")}
     </select></div>
```

```
    <div class="field full"><label>Resume file</label><input name="resumeFile" type="file"><span class="help">Current: ${esc(c.resume||"No resume recorded")}. Demo saves only the filename.</span></div>
    <div class="field full"><label class="checkline"><input type="checkbox" name="certified" ${c.certified?"checked":""}> Certification verified in the internal register</label></div>
  </div>
 </form>`;
 setModal(isNew?"Add Talent Pool Record":"Edit Talent Pool Record",body,[
  {label:"Cancel",action:"closeModal()"},
  {label:"Save record",kind:"primary",action:"document.getElementById('candidateForm').requestSubmit()"}
 ]);
}

function saveCandidate(event,oldId){
 event.preventDefault();
 const f=new FormData(event.target);
 const cid=String(f.get("id")||"").trim();
 if(!cid){toast("Candidate ID is required");return;}
 const old=oldId?getCandidate(oldId):null;
 if(!old&&db.candidates.some(c=>c.id===cid)){toast("Candidate ID already exists");return;}
 const c=old||{id:cid,history:[],retention:{}};
 for(const key of ["name","district","qualification","skill","scheme","batch","certificate","phone","email","visibility","status"]) c[key]=String(f.get(key)||"").trim();
 c.certified=f.get("certified")==="on";
 const file=f.get("resumeFile");
 if(file&&file.name)c.resume=file.name;
 if(!old)db.candidates.unshift(c);
 logAction(old?"Update talent record":"Create talent record",cid,c.name);
 closeModal();render();toast("Candidate record saved");
}
```

```
function viewCandidate(cid){
 const c=getCandidate(cid);
 if(!c)return;
 const body=`<div class="actions" style="margin-bottom:14px">
  ${btn("Edit record",`closeModal();candidateForm('${cid}')`)}
  ${btn(c.visibility==="Hidden"?"Make visible to verified employers":"Hide from employers",`toggleVisibility('${cid}')`)}
  ${btn("Record interview",`closeModal();interviewForm('${cid}')`)}
 </div>
 <div class="detail-list">
  ${[
   ["Candidate ID",c.id],["Name",c.name],["District",c.district],
   ["Qualification",c.qualification],["Skill / Trade",c.skill],
   ["Scheme",c.scheme],["Batch",c.batch],["Certificate",c.certificate],
   ["Certified",c.certified?"Yes":"No"],["Contact",c.phone],["Email",c.email],
   ["Resume",c.resume],["Visibility",c.visibility],["Status",c.status],
   ["Employer",employerName(c.employerId)],["Vacancy",jobTitle(c.jobId)],
   ["Monthly salary",c.salary?money(c.salary):"—"],["Placed date",fmtDate(c.placedAt)],
   ["Employment proof",c.proof],["Proof status",c.proofStatus],
   ["Manager sanction",c.sanctionStatus]
  ].map(([k,v])=>detailItem(k,v)).join("")}
 </div>
 <div class="note">Candidate registration and public profile changes are expected to be handled through the KASE website. This internal screen manages the candidate record and placement workflow.</div>`;
 setModal("Candidate record – "+c.name,body,[{label:"Close",action:"closeModal()"}]);
}

function toggleVisibility(cid){
 const c=getCandidate(cid);
 c.visibility=c.visibility==="Hidden"?"Visible to verified employers":"Hidden";
 logAction("Change talent visibility",cid,c.visibility);
```

```
  saveDb();closeModal();render();toast("Candidate visibility updated");
}

function renderEmployers(){
  const q=($("globalSearch").value||"").toLowerCase();
  const rows=db.employers.filter(e=>JSON.stringify(e).toLowerCase().includes(q));
  return pageHead("Employer Register","KASE ERP › Placements › Employers",btn("Register employer","employerForm()","primary")+btn("Export CSV","exportRows('employers')"))+
  `<div class="note">Employer accounts and registration are managed through the KASE website. This register is for internal scrutiny, verification and employer record management.</div>
  <div class="filters">
    <input placeholder="Search employer or district" oninput="filterEmployers(this.value)" value="${esc(q)}">
    <select id="employerStatusFilter" onchange="filterEmployers(document.querySelector('.filters input').value)">
      <option value="">All verification statuses</option><option>Pending Verification</option><option>Verified</option><option>Rejected</option>
    </select>
  </div>
  <div id="employerTable">${employerTable(rows)}</div>`;
}

function employerTable(rows){
  return table(["Employer ID","Organisation","Sector","District","Contact","Verification","Actions"],rows,e=>[
    esc(e.id),
    `<button class="link-btn" onclick="viewEmployer('${e.id}')">${esc(e.name)}</button>`,
    esc(e.sector),esc(e.district),esc(e.contact),statusBadge(e.status),
    btn("Review",`viewEmployer('${e.id}')`,"small")
  ]);
}

function filterEmployers(value){
```

```
  const q=value.toLowerCase(),s=$("employerStatusFilter")?.value||"";

$("employerTable").innerHTML=employerTable(db.employers.filter(e=>JSON.stringify(e).toLowerCase().includes(q)&&(!s||e.status===s)));
}

function employerForm(eid=""){
  const e=eid?getEmployer(eid):{
    id:id("EMP"),name:"",sector:"",district:"",contact:"",email:"",
    phone:"",status:"Pending Verification",notes:""
  };
  const body=`<form id="employerForm" onsubmit="saveEmployer(event,'${esc(eid)}')">
    <div class="form-grid">
      <div class="field"><label>Employer ID</label><input name="id" value="${esc(e.id)}" readonly></div>
      <div class="field"><label>Organisation name</label><input name="name" value="${esc(e.name)}" required></div>
      <div class="field"><label>Sector</label><input name="sector" value="${esc(e.sector)}" required></div>
      <div class="field"><label>District</label><input name="district" value="${esc(e.district)}" required></div>
      <div class="field"><label>Contact person / section</label><input name="contact" value="${esc(e.contact)}"></div>
      <div class="field"><label>Email</label><input name="email" type="email" value="${esc(e.email)}"></div>
      <div class="field"><label>Telephone</label><input name="phone" value="${esc(e.phone)}"></div>
      <div class="field"><label>Verification status</label><select name="status">${["Pending Verification","Verified","Rejected"].map(s=>`<option ${e.status===s?"selected":""}>${s}</option>`).join("")}</select></div>
      <div class="field full"><label>Internal notes / scrutiny remarks</label><textarea name="notes">${esc(e.notes)}</textarea></div>
    </div>
  </form>`;
  setModal(eid?"Edit employer record":"Register employer record",body,[
```

```
    {label:"Cancel",action:"closeModal()"},

{label:"Save",kind:"primary",action:"document.getElementById('employerForm').requestSubmit()"}

  ]);

}


function saveEmployer(event,eid){

  event.preventDefault();

  const f=new FormData(event.target);

  const e=eid?getEmployer(eid):{id:String(f.get("id")),verifiedBy:"",verifiedAt:""};

  for(const k of
["name","sector","district","contact","email","phone","status","notes"])e[k]=String(f.get(k)||"").trim
();


if(e.status==="Verified"&&!e.verifiedAt){e.verifiedBy=accounts[loggedInId].label;e.verifiedAt=isoTod
ay();}

  if(!eid)db.employers.unshift(e);

  logAction(eid?"Update employer":"Register employer",e.id,e.name);

  closeModal();render();toast("Employer record saved");

}


function viewEmployer(eid){

  const e=getEmployer(eid);

  if(!e)return;

  const body=`${recordDetails("Employer",e)}

    <div class="note">Review submitted organisation details and supporting evidence according to
KASE verification procedures before marking an employer as verified. Employer access is managed
separately through the website.</div>

    <div class="field"><label>Verification decision</label><select
id="employerDecision"><option>Pending
Verification</option><option>Verified</option><option>Rejected</option></select></div>

    <div class="field"><label>Review remarks</label><textarea
id="employerReviewNotes">${esc(e.notes||"")}</textarea></div>`;

  setModal("Employer verification – "+e.name,body,[
```

```
    {label:"Close",action:"closeModal()"},
    {label:"Save decision",kind:"primary",action:`saveEmployerDecision('${eid}')`}
  ]);
  $("employerDecision").value=e.status;
}

function saveEmployerDecision(eid){
  const e=getEmployer(eid);
  const decision=$("employerDecision").value;
  e.status=decision;
  e.notes=$("employerReviewNotes").value.trim();
  e.verifiedBy=decision==="Verified"?accounts[loggedInId].label:"";
  e.verifiedAt=decision==="Verified"?isoToday():"";
  logAction("Employer verification decision",eid,decision);
  saveDb();closeModal();render();toast("Employer verification updated");
}

function renderVacancies(){
  const q=($("globalSearch").value||"").toLowerCase();
  const rows=db.jobs.filter(j=>JSON.stringify(j).toLowerCase().includes(q));
  return pageHead("Vacancies & Matching","KASE ERP › Placements › Vacancies",btn("Add vacancy","jobForm()","primary")+btn("Export CSV","exportRows('jobs')"))+
  `<div class="note">Vacancies received from verified employers through the website may be synchronised into this register. Internal staff can review vacancies, manage status and view matching certified candidates.</div>`+
  table(["Vacancy","Employer","District","Skill","Positions","Monthly pay","Deadline","Status","Actions"],rows,j=>[
    `<button class="link-btn" onclick="viewJob('${j.id}')">${esc(j.title)}</button>`,
    esc(employerName(j.employerId)),esc(j.district),esc(j.skill),j.positions,
    money(j.salary),fmtDate(j.deadline),statusBadge(j.status),
    btn("Matches",`showMatches('${j.id}')`,"small")+" "+btn("Edit",`jobForm('${j.id}')`,"small")
  ]);
```

```
}

function jobForm(jid=""){
  const j=jid?getJob(jid):{
    id:id("JOB"),employerId:db.employers.find(e=>e.status==="Verified")?.id||"",
    title:"",district:"",skill:"",qualification:"",positions:1,salary:0,
    workMode:"On-site",postedAt:isoToday(),deadline:daysAfter(isoToday(),30),
    status:"Open",description:""
  };
  const body=`<form id="jobForm" onsubmit="saveJob(event,'${esc(jid)}')">
    <div class="form-grid">
      <div class="field"><label>Vacancy ID</label><input name="id" value="${esc(j.id)}" readonly></div>
      <div class="field"><label>Employer</label><select name="employerId" required>${employerOptions(j.employerId)}</select></div>
      <div class="field"><label>Vacancy title</label><input name="title" value="${esc(j.title)}" required></div>
      <div class="field"><label>District</label><input name="district" value="${esc(j.district)}" required></div>
      <div class="field"><label>Required skill / trade</label><input name="skill" value="${esc(j.skill)}" required></div>
      <div class="field"><label>Minimum qualification</label><input name="qualification" value="${esc(j.qualification)}"></div>
      <div class="field"><label>Number of positions</label><input name="positions" type="number" min="1" value="${esc(j.positions)}" required></div>
      <div class="field"><label>Monthly salary (Rs.)</label><input name="salary" type="number" min="0" value="${esc(j.salary)}"></div>
      <div class="field"><label>Work mode</label><select name="workMode">${["On-site","Hybrid","Remote"].map(s=>`<option ${j.workMode===s?"selected":""}>${s}</option>`).join("")}</select></div>
      <div class="field"><label>Application deadline</label><input name="deadline" type="date" value="${esc(j.deadline)}"></div>
      <div class="field"><label>Status</label><select name="status">${["Draft","Open","On Hold","Filled","Closed"].map(s=>`<option ${j.status===s?"selected":""}>${s}</option>`).join("")}</select></div>
```

```
    <div class="field full"><label>Role description</label><textarea
name="description">${esc(j.description)}</textarea></div>
  </div>
 </form>`;
 setModal(jid?"Edit vacancy":"Add vacancy",body,[
  {label:"Cancel",action:"closeModal()"},
  {label:"Save
vacancy",kind:"primary",action:"document.getElementById('jobForm').requestSubmit()"}
 ]);
}

function saveJob(event,jid){
 event.preventDefault();
 const f=new FormData(event.target);
 const j=jid?getJob(jid):{id:String(f.get("id")),postedAt:isoToday()};
 for(const k of
["employerId","title","district","skill","qualification","workMode","deadline","status","description"])j
[k]=String(f.get(k)||"").trim();
 j.positions=Number(f.get("positions")||1);
 j.salary=Number(f.get("salary")||0);
 if(getEmployer(j.employerId)?.status!=="Verified"&&j.status==="Open"){
  toast("Only verified employers can have open vacancies");
  return;
 }
 if(!jid)db.jobs.unshift(j);
 logAction(jid?"Update vacancy":"Create vacancy",j.id,j.title);
 closeModal();render();toast("Vacancy saved");
}

function viewJob(jid){
 const j=getJob(jid);
 if(!j)return;
```

```
 setModal("Vacancy – "+j.title,recordDetails("Vacancy",{...j,employer:employerName(j.employerId)}),[
   {label:"Close",action:"closeModal()"},
   {label:"View matching candidates",kind:"primary",action:`showMatches('${jid}')`}
 ]);
}

function showMatches(jid){
 const j=getJob(jid);
 if(!j)return;
 const matches=db.candidates.filter(c=>c.certified&&c.visibility!=="Hidden"&&c.status!=="Placed"&&c.skill.toLowerCase().includes(j.skill.toLowerCase())&&(j.district===""||c.district.toLowerCase()===j.district.toLowerCase()));
 const body=`<div class="note">Matching uses the recorded skill and district as simple prototype rules. Production matching should use the approved eligibility and selection criteria.</div>`+
   table(["Candidate","District","Skill","Qualification","Status","Action"],matches,c=>[
     `<button class="link-btn" onclick="viewCandidate('${c.id}')">${esc(c.name)}</button>`,
     esc(c.district),esc(c.skill),esc(c.qualification),statusBadge(c.status),
     btn("Add to interview",`closeModal();interviewForm('${c.id}','${jid}')`,"small")
   ],"No matching candidates found.");
 setModal("Candidate matches – "+j.title,body,[{label:"Close",action:"closeModal()"}]);
}

function renderDrives(){
 return pageHead("Placement Drives","KASE ERP › Placements › Drives",btn("Schedule drive","driveForm()","primary")+btn("Export CSV","exportRows('drives')"))+
 `<div class="note">Drive invitations, candidate responses and admit-card issue are normally managed through the website. The internal module records the drive, eligible candidate list and attendance status.</div>`+
 table(["Drive","Employer","Vacancy","Date","Venue / mode","Candidates","Status","Actions"],db.drives,d=>[
   `<button class="link-btn" onclick="viewDrive('${d.id}')">${esc(d.title)}</button>`,
   esc(employerName(d.employerId)),esc(jobTitle(d.jobId)),fmtDate(d.date),
```

```
    esc(d.venue+" / "+d.mode),d.candidateIds.length,statusBadge(d.status),
    btn("Manage",`viewDrive('${d.id}')`,"small")+" "+btn("Edit",`driveForm('${d.id}')`,"small")
  ]);
}

function driveForm(did=""){
  const d=did?getDrive(did):{
    id:id("DRV"),title:"",
    employerId:db.employers.find(e=>e.status==="Verified")?.id||"",
    jobId:db.jobs.find(j=>j.status==="Open")?.id||"",
    date:daysAfter(isoToday(),7),venue:"",mode:"In-person",status:"Scheduled",
    candidateIds:[],attendance:{},createdAt:isoToday()
  };
  const body=`<form id="driveForm" onsubmit="saveDrive(event,'${esc(did)}')">
    <div class="form-grid">
      <div class="field"><label>Drive ID</label><input name="id" value="${esc(d.id)}" readonly></div>
      <div class="field"><label>Drive title</label><input name="title" value="${esc(d.title)}"
required></div>
      <div class="field"><label>Employer</label><select name="employerId"
required>${employerOptions(d.employerId)}</select></div>
      <div class="field"><label>Vacancy</label><select name="jobId"
required>${jobOptions(d.jobId)}</select></div>
      <div class="field"><label>Date</label><input name="date" type="date" value="${esc(d.date)}"
required></div>
      <div class="field"><label>Venue / meeting details</label><input name="venue"
value="${esc(d.venue)}" required></div>
      <div class="field"><label>Mode</label><select name="mode">${["In-
person","Online","Hybrid"].map(s=>`<option
${d.mode===s?"selected":""}>${s}</option>`).join("")}</select></div>
      <div class="field"><label>Status</label><select
name="status">${["Draft","Scheduled","Completed","Cancelled"].map(s=>`<option
${d.status===s?"selected":""}>${s}</option>`).join("")}</select></div>
      <div class="field full"><label>Eligible candidates (select multiple)</label>
```

```
      <select name="candidateIds" multiple
size="6">${db.candidates.filter(c=>c.certified&&c.status!=="Placed").map(c=>`<option
value="${esc(c.id)}" ${d.candidateIds.includes(c.id)?"selected":""}>${esc(c.id+" – "+c.name+" /
"+c.skill)}</option>`).join("")}</select>

      <span class="help">Hold Ctrl to select more than one candidate.</span>

    </div>

   </div>

 </form>`;

 setModal(did?"Edit placement drive":"Schedule placement drive",body,[

  {label:"Cancel",action:"closeModal()"},

  {label:"Save
drive",kind:"primary",action:"document.getElementById('driveForm').requestSubmit()"}

 ]);

}


function saveDrive(event,did){

 event.preventDefault();

 const f=new FormData(event.target);

 const d=did?getDrive(did):{id:String(f.get("id")),attendance:{},createdAt:isoToday()};

 for(const k of
["title","employerId","jobId","date","venue","mode","status"])d[k]=String(f.get(k)||"").trim();

 d.candidateIds=f.getAll("candidateIds");

 if(!did)db.drives.unshift(d);

 d.candidateIds.forEach(cid=>{if(!d.attendance[cid])d.attendance[cid]="Invited";});

 logAction(did?"Update placement drive":"Create placement drive",d.id,d.title);

 closeModal();render();toast("Placement drive saved");

}


function viewDrive(did){

 const d=getDrive(did);

 if(!d)return;

 const candidates=d.candidateIds.map(cid=>getCandidate(cid)).filter(Boolean);
```

```
  const
body=`${recordDetails("Drive",{...d,employer:employerName(d.employerId),vacancy:jobTitle(d.jobId
),candidateCount:d.candidateIds.length})}

   <div class="card" style="margin-top:15px">

    <div class="card-head"><h3>Invited candidates and attendance</h3>${btn("Print drive
sheet",`printDrive('${did}')`)}</div>

    ${table(["Candidate","Skill","Invitation / RSVP","Attendance","Admit card"],candidates,c=>[

      `<button class="link-btn" onclick="viewCandidate('${c.id}')">${esc(c.name)}</button>`,

      esc(c.skill),statusBadge(d.attendance[c.id]||"Invited"),

      `<select id="att_${esc(c.id)}">

        <option ${d.attendance[c.id]==="Invited"?"selected":""}>Invited</option>

        <option ${d.attendance[c.id]==="Accepted"?"selected":""}>Accepted</option>

        <option ${d.attendance[c.id]==="Declined"?"selected":""}>Declined</option>

        <option ${d.attendance[c.id]==="Present"?"selected":""}>Present</option>

        <option ${d.attendance[c.id]==="Absent"?"selected":""}>Absent</option>

      </select>`,

      `<button class="btn small" onclick="admitCard('${did}','${c.id}')">Preview</button>`

    ],"No candidates have been assigned to this drive.")}

   </div>

   <div class="note">The invitation/RSVP and admit-card workflow is represented for internal
tracking. Candidate-facing notices and QR validation must be integrated with the website for live
operation.</div>`;

  setModal("Placement drive – "+d.title,body,[

   {label:"Close",action:"closeModal()"},

   {label:"Save attendance",kind:"primary",action:`saveDriveAttendance('${did}')`}

  ]);

}


function saveDriveAttendance(did){

  const d=getDrive(did);

  d.candidateIds.forEach(cid=>{

   const el=$("att_"+cid);

   if(el)d.attendance[cid]=el.value;
```

```
  });
  logAction("Update drive attendance",did,d.title);
  saveDb();closeModal();render();toast("Drive attendance updated");
}

function admitCard(did,cid){
  const d=getDrive(did),c=getCandidate(cid);
  setModal("Admit card preview",`<div class="detail-list">${
    [["Candidate",c.name],["Candidate ID",c.id],["Drive",d.title],["Drive ID",d.id],
     ["Employer",employerName(d.employerId)],["Vacancy",jobTitle(d.jobId)],
     ["Date",fmtDate(d.date)],["Venue / mode",d.venue+" / "+d.mode],
     ["Admit card reference","AC-"+d.id+"-"+c.id]
    ].map(([k,v])=>detailItem(k,v)).join("")
  }</div><p class="muted">Preview only. Website integration is required to issue candidate admit cards and scan QR codes.</p>`,[
    {label:"Close",action:"closeModal()"},
    {label:"Print",kind:"primary",action:"window.print()"}
  ]);
}

function printDrive(did){window.print();}

function renderInterviews(){
  return pageHead("Interview Pipeline","KASE ERP › Placements › Interview Pipeline",btn("Record interview / update","interviewForm()","primary")+btn("Export CSV","exportRows('interviews')"))+
  `<div class="note">Employer interview feedback may be received through the website and reflected here. Internal staff can maintain the interview stage, schedule and follow-up notes.</div>`+
  table(["Reference","Candidate","Employer","Vacancy","Interview date","Stage","Employer update","Actions"],db.interviews,i=>[
    esc(i.id),
    `<button class="link-btn" onclick="viewCandidate('${i.candidateId}')">${esc(candidateName(i.candidateId))}</button>`,
```

```
    esc(employerName(i.employerId)),esc(jobTitle(i.jobId)),fmtDate(i.date),
    statusBadge(i.stage),esc(i.employerUpdate||"—"),
    btn("Update",`interviewForm('${i.candidateId}','${i.jobId}','${i.id}')`,"small")
  ]);
}

function interviewForm(cid="",jid="",iid=""){
  const i=iid?db.interviews.find(x=>x.id===iid):{
    id:id("INT"),candidateId:cid||db.candidates[0]?.id||"",
    employerId:getJob(jid)?.employerId||db.employers[0]?.id||"",
    jobId:jid||db.jobs[0]?.id||"",driveId:"",stage:"Invited",
    date:isoToday(),employerUpdate:"",updatedAt:isoToday(),notes:""
  };
  const body=`<form id="interviewForm" onsubmit="saveInterview(event,'${esc(iid)}')">
    <div class="form-grid">
      <div class="field"><label>Interview reference</label><input name="id" value="${esc(i.id)}" readonly></div>
      <div class="field"><label>Candidate</label><select name="candidateId" required>${candidateOptions(i.candidateId)}</select></div>
      <div class="field"><label>Employer</label><select name="employerId" required>${employerOptions(i.employerId)}</select></div>
      <div class="field"><label>Vacancy</label><select name="jobId" required>${jobOptions(i.jobId)}</select></div>
      <div class="field"><label>Placement drive (optional)</label><select name="driveId"><option value="">No drive</option>${driveOptions(i.driveId)}</select></div>
      <div class="field"><label>Interview date</label><input type="date" name="date" value="${esc(i.date)}" required></div>
      <div class="field"><label>Pipeline stage</label><select name="stage">${["Invited","Interview Scheduled","Interview Completed","Shortlisted","Offer Made","Offer Accepted","Not Selected","Withdrawn"].map(s=>`<option ${i.stage===s?"selected":""}>${s}</option>`).join("")}</select></div>
      <div class="field"><label>Employer update / feedback</label><input name="employerUpdate" value="${esc(i.employerUpdate)}"></div>
      <div class="field full"><label>Internal follow-up notes</label><textarea name="notes">${esc(i.notes)}</textarea></div>
```

```
  </div>
 </form>`;
 setModal(iid?"Update interview pipeline":"Record interview pipeline",body,[
  {label:"Cancel",action:"closeModal()"},
  {label:"Save interview",kind:"primary",action:"document.getElementById('interviewForm').requestSubmit()"}
 ]);
}

function saveInterview(event,iid){
 event.preventDefault();
 const f=new FormData(event.target);
 const i=iid?db.interviews.find(x=>x.id===iid):{id:String(f.get("id"))};
 for(const k of ["candidateId","employerId","jobId","driveId","date","stage","employerUpdate","notes"])i[k]=String(f.get(k)||"").trim();
 i.updatedAt=isoToday();
 if(!iid)db.interviews.unshift(i);
 const c=getCandidate(i.candidateId);
 if(c&&["Invited","Interview Scheduled","Interview Completed","Shortlisted","Offer Made","Offer Accepted"].includes(i.stage)&&c.status==="Available")c.status="Interviewing";
 if(c&&i.stage==="Not Selected"&&c.status==="Interviewing")c.status="Available";
 if(i.stage==="Offer Accepted"&&c)c.status="Awaiting Verification";
 logAction(iid?"Update interview":"Create interview",i.id,i.stage);
 closeModal();render();toast("Interview pipeline saved");
}

function renderVerification(){
 const rows=db.candidates.filter(c=>c.status==="Awaiting Verification"||c.proofStatus==="Submitted"||c.proofStatus==="Under Review"||c.proofStatus==="Verified");
 return pageHead("Employment Verification & Sanction","KASE ERP › Placements › Verification",btn("Register employment proof","proofForm()","primary")+btn("Export CSV","exportRows('verification')"))+
```

```
  `<div class="note">Employment evidence is uploaded by candidates through the website. Executive (Placements) scrutinises the evidence; Manager (Placements) records the sanction decision. File names shown in this prototype are not actual stored documents.</div>`+
  table(["Candidate","Employer","Vacancy","Placed date","Monthly salary","Proof file","Verification","Sanction","Actions"],rows,c=>[
    `<button class="link-btn" onclick="viewCandidate('${c.id}')">${esc(c.name)}</button>`,
    esc(employerName(c.employerId)),esc(jobTitle(c.jobId)),fmtDate(c.placedAt),
    c.salary?money(c.salary):"—",esc(c.proof||"Not submitted"),
    statusBadge(c.proofStatus),statusBadge(c.sanctionStatus),
    btn("Review",`viewVerification('${c.id}')`,"small")
  ],"No employment proof is awaiting review.");
}

function proofForm(cid=""){
  const c=cid?getCandidate(cid):db.candidates.find(x=>x.status==="Awaiting Verification")||db.candidates[0];
  if(!c){toast("No candidate records available");return;}
  const body=`<form id="proofForm" onsubmit="saveProof(event)">
    <div class="field"><label>Candidate</label><select name="candidateId">${candidateOptions(c.id)}</select></div>
    <div class="form-grid">
      <div class="field"><label>Employer</label><select name="employerId">${employerOptions(c.employerId||"")}</select></div>
      <div class="field"><label>Vacancy</label><select name="jobId">${jobOptions(c.jobId||"")}</select></div>
      <div class="field"><label>Employment start date</label><input name="placedAt" type="date" value="${esc(c.placedAt||isoToday())}" required></div>
      <div class="field"><label>Monthly salary (Rs.)</label><input name="salary" type="number" min="0" value="${esc(c.salary||"")}" required></div>
      <div class="field full"><label>Employment proof file</label><input name="proofFile" type="file" required><span class="help">Appointment letter, employer confirmation or other accepted proof. This prototype records the selected filename only.</span></div>
    </div>
  </form>`;
  setModal("Register employment proof",body,[
```

```
  {label:"Cancel",action:"closeModal()"},
  {label:"Submit for verification",kind:"primary",action:"document.getElementById('proofForm').requestSubmit()"}
 ]);
}

function saveProof(event){
 event.preventDefault();
 const f=new FormData(event.target);
 const cid=String(f.get("candidateId")),c=getCandidate(cid),file=f.get("proofFile");
 if(!file||!file.name){toast("Select an employment proof file");return;}
 c.employerId=String(f.get("employerId"));
 c.jobId=String(f.get("jobId"));
 c.placedAt=String(f.get("placedAt"));
 c.salary=Number(f.get("salary")||0);
 c.proof=file.name;
 c.proofStatus="Submitted";
 c.sanctionStatus="Pending";
 c.status="Awaiting Verification";
 if(!c.retention)c.retention={};
 logAction("Employment proof submitted",cid,file.name);
 closeModal();render();toast("Employment proof recorded for verification");
}

function viewVerification(cid){
 const c=getCandidate(cid);
 if(!c)return;
 const body=`<div class="detail-list">${
  [["Candidate",c.name],["Employer",employerName(c.employerId)],
   ["Vacancy",jobTitle(c.jobId)],["Employment start date",fmtDate(c.placedAt)],
   ["Monthly salary",money(c.salary)],["Employment proof file",c.proof||"Not submitted"],
```

```
    ["Proof verification",c.proofStatus],["Manager sanction",c.sanctionStatus]
   ].map(([k,v])=>detailItem(k,v)).join("")
  }</div>
  <div class="field" style="margin-top:14px"><label>Executive verification decision</label><select id="proofDecision">
   <option>Submitted</option><option>Under Review</option><option>Verified</option><option>Rejected</option>
  </select></div>
  <div class="field"><label>Verification remarks</label><textarea id="proofRemarks">${esc(c.proofRemarks||"")}</textarea></div>
  ${canManage()?`<div class="field"><label>Manager sanction decision</label><select id="sanctionDecision"><option>Pending</option><option>Sanctioned</option><option>Rejected</option></select></div><div class="field"><label>Sanction remarks</label><textarea id="sanctionRemarks">${esc(c.sanctionRemarks||"")}</textarea></div>`:`<div class="note">Manager sanction is available only to the Manager (Placements) account.</div>`}`;
  setModal("Employment evidence review – "+c.name,body,[
   {label:"Close",action:"closeModal()"},
   {label:"Save review",kind:"primary",action:`saveVerificationDecision('${cid}')`}
  ]);
  $("proofDecision").value=c.proofStatus||"Submitted";
  if(canManage())$("sanctionDecision").value=c.sanctionStatus||"Pending";
}

function saveVerificationDecision(cid){
  const c=getCandidate(cid),decision=$("proofDecision").value;
  c.proofStatus=decision;
  c.proofRemarks=$("proofRemarks").value.trim();
  if(decision==="Verified"){
   c.status="Placed";
   if(!c.placedAt)c.placedAt=isoToday();
  }else if(decision==="Rejected"){
   c.status="Awaiting Verification";
  }
```

```
  if(canManage()){
    c.sanctionStatus=$("sanctionDecision").value;
    c.sanctionRemarks=$("sanctionRemarks").value.trim();
    if(c.sanctionStatus==="Sanctioned"&&c.proofStatus==="Verified"){
      c.status="Retention Monitoring";
      if(!c.retention)c.retention={};
      for(const day of [30,60,90])if(!c.retention[day])c.retention[day]={status:"Due",proof:"",date:""};
    }
  }
  logAction("Employment proof review",cid,decision+(canManage()?" / "+c.sanctionStatus:""));
  saveDb();closeModal();render();toast("Employment review updated");
}

function renderRetention(){
  const rows=db.candidates.filter(c=>c.status==="Retention Monitoring"||c.status==="Placed").sort((a,b)=>(a.placedAt||"").localeCompare(b.placedAt||""));
  const totalDue=rows.reduce((n,c)=>n+[30,60,90].filter(d=>milestoneStatus(c,d)==="Due"||milestoneStatus(c,d)==="Pending").length,0);
  return pageHead("Retention Milestones","KASE ERP › Placements › Retention",btn("Refresh milestone dates","refreshRetention()")+btn("Export CSV","exportRows('retention')"))+
  `<div class="cards">
    ${metric("Candidates under monitoring",rows.length,"Employment proof sanctioned")}
    ${metric("Open milestones",totalDue,"30, 60 and 90-day checks")}
    ${metric("Milestones verified",rows.reduce((n,c)=>n+[30,60,90].filter(d=>milestoneStatus(c,d)==="Verified").length,0),"Proof reviewed")}
    ${metric("Manager account",canManage()?"Active":"Read-only","Sanction workflow")}
  </div>
  <div class="note">Milestones are calculated from the recorded employment start date. Supporting evidence should be submitted through the website and checked internally. Confirm milestone rules for the applicable scheme before operational use.</div>`+
  table(["Candidate","Employer","Employment date","30 days","60 days","90 days","Actions"],rows,c=>[
```

```
    `<button class="link-btn" onclick="viewCandidate('${c.id}')">${esc(c.name)}</button>`,
    esc(employerName(c.employerId)),fmtDate(c.placedAt),
    milestoneCell(c,30),milestoneCell(c,60),milestoneCell(c,90),
    btn("Manage proof",`viewRetention('${c.id}')`,"small")
  ],"No candidates are currently under retention monitoring.");
}

function milestoneStatus(c,day){
  const m=c.retention?.[day];
  if(m?.status)return m.status;
  if(!c.placedAt)return "Pending";
  return new Date(isoToday()+"T00:00:00")>=new
Date(daysAfter(c.placedAt,day)+"T00:00:00")?"Due":"Not Due";
}
function milestoneCell(c,day){
  const due=daysAfter(c.placedAt,day);
  return `${statusBadge(milestoneStatus(c,day))}<div class="muted" style="font-size:11px;margin-
top:4px">${due?fmtDate(due):"Date pending"}</div>`;
}

function viewRetention(cid){
  const c=getCandidate(cid);
  if(!c.retention)c.retention={};
  const body=`<div class="detail-list">${
    [["Candidate",c.name],["Employer",employerName(c.employerId)],
     ["Employment start date",fmtDate(c.placedAt)],["Monthly salary",money(c.salary)]
    ].map(([k,v])=>detailItem(k,v)).join("")
  }</div>
  <div class="card" style="margin-top:14px">
    <h3>Retention evidence</h3>
    ${[30,60,90].map(day=>{
      const m=c.retention[day]||{status:milestoneStatus(c,day),proof:"",date:""};
```

```
    return `<div style="border-top:1px solid var(--line);padding:12px 0">
      <h3>${day}-day milestone</h3>
      <p>Due date: ${fmtDate(daysAfter(c.placedAt,day))} · Current status: ${esc(m.status||milestoneStatus(c,day))}</p>
      <div class="field"><label>Evidence filename / reference</label><input id="retProof${day}" value="${esc(m.proof||"")}" placeholder="Salary slip or employer confirmation reference"></div>
      <div class="field"><label>Review status</label><select id="retStatus${day}">
        ${["Not Due","Due","Submitted","Under Review","Verified","Rejected","Pending"].map(s=>`<option ${(m.status||milestoneStatus(c,day))===s?"selected":""}>${s}</option>`).join("")}
      </select></div>
    </div>`;
  }).join("")}
 </div>
 ${canManage()?`<div class="note">As Manager (Placements), you may sanction the FICO milestone after required retention evidence has been verified.</div>`:""}`;
 setModal("Retention evidence – "+c.name,body,[
  {label:"Close",action:"closeModal()"},
  {label:"Save milestone records",kind:"primary",action:`saveRetention('${cid}')`}
 ]);
}

function saveRetention(cid){
 const c=getCandidate(cid);
 if(!c.retention)c.retention={};
 for(const day of [30,60,90]){
  const proof=$("retProof"+day).value.trim(),status=$("retStatus"+day).value;
  c.retention[day]={proof,status,date:status==="Verified"?isoToday():(c.retention[day]?.date||"")};
 }
 logAction("Update retention milestones",cid,c.name);
 saveDb();closeModal();render();toast("Retention records saved");
}
```

```
function refreshRetention(){
 db.candidates.forEach(c=>{
  if(c.status==="Retention Monitoring"&&!c.retention)c.retention={};
 });
 saveDb();render();toast("Retention dates refreshed");
}

function renderFico(){
 return pageHead("FICO & eOffice","KASE ERP › Placements › FICO and AP Queue",btn("Create FICO request","ficoForm()","primary")+btn("Export CSV","exportRows('fico')"))+
 `<div class="note">This register records milestone reports, approved eOffice proceedings and the Accounts Payable (AP) queue. It does not create a live FICO transaction or eOffice file in this browser prototype.</div>`+
 table(["Request","Candidate","Milestone","Amount","FICO report","eOffice reference","AP status","Actions"],db.fico,q=>[
  esc(q.id),esc(candidateName(q.candidateId)),esc(q.milestone),money(q.amount),
  esc(q.report||"Not attached"),esc(q.eoffice||"Not entered"),
  statusBadge(q.status),btn("Review",`viewFico('${q.id}')`,"small")
 ]);
}

function ficoForm(){
 const eligible=db.candidates.filter(c=>c.status==="Retention Monitoring"&&c.sanctionStatus==="Sanctioned");
 const body=`<form id="ficoForm" onsubmit="saveFico(event)">
  <div class="form-grid">
   <div class="field"><label>Candidate</label><select name="candidateId" required>${candidateOptions(eligible[0]?.id||db.candidates[0]?.id||"")}</select></div>
   <div class="field"><label>Milestone</label><select name="milestone">${["Placement","30-day retention","60-day retention","90-day retention"].map(s=>`<option>${s}</option>`).join("")}</select></div>
   <div class="field"><label>Amount (Rs.)</label><input type="number" name="amount" min="1" required></div>
```

```
    <div class="field"><label>Budget ledger / head</label><input name="ledger" required placeholder="Enter applicable ledger / head"></div>
    <div class="field"><label>Approved eOffice proceeding reference</label><input name="eoffice" required placeholder="File / proceeding number"></div>
    <div class="field"><label>FICO milestone report file</label><input type="file" name="reportFile"><span class="help">The demo stores the filename only.</span></div>
   </div>
 </form>`;
 setModal("Create FICO request",body,[
  {label:"Cancel",action:"closeModal()"},
  {label:"Add to AP queue",kind:"primary",action:"document.getElementById('ficoForm').requestSubmit()"}
 ]);
}

function saveFico(event){
 event.preventDefault();
 const f=new FormData(event.target),file=f.get("reportFile");
 const q={
  id:id("FICO"),candidateId:String(f.get("candidateId")),
  milestone:String(f.get("milestone")),amount:Number(f.get("amount")),
  ledger:String(f.get("ledger")).trim(),eoffice:String(f.get("eoffice")).trim(),
  report:file&&file.name?file.name:"",
  status:"Submitted for AP Processing",createdAt:isoToday(),history:[]
 };
 if(!q.eoffice||!q.ledger||q.amount<=0){
  toast("Amount, ledger and eOffice reference are required");
  return;
 }
 db.fico.unshift(q);
 logAction("Create FICO / AP request",q.id,q.milestone);
 closeModal();render();toast("FICO request added to AP queue");
```

```
}

function viewFico(qid){
 const q=db.fico.find(x=>x.id===qid);
 const body=`${recordDetails("FICO request",{...q,candidate:candidateName(q.candidateId),createdAt:fmtDate(q.createdAt)})}
   <div class="field" style="margin-top:14px"><label>AP queue status</label><select id="ficoStatus">
    ${["Draft","Report Required","Submitted for AP Processing","Under AP Review","Approved for Payment","Paid","Returned for Correction","Rejected"].map(s=>`<option ${q.status===s?"selected":""}>${s}</option>`).join("")}
   </select></div>
   <div class="field"><label>Internal remarks</label><textarea id="ficoRemarks">${esc(q.remarks||"")}</textarea></div>`;
 setModal("FICO request – "+q.id,body,[
   {label:"Close",action:"closeModal()"},
   {label:"Save status",kind:"primary",action:`saveFicoStatus('${qid}')`}
 ]);
}

function saveFicoStatus(qid){
 const q=db.fico.find(x=>x.id===qid);
 q.status=$("ficoStatus").value;
 q.remarks=$("ficoRemarks").value.trim();
 q.history=q.history||[];
 q.history.unshift({date:new Date().toLocaleString("en-GB"),user:accounts[loggedInId].label,status:q.status,remarks:q.remarks});
 logAction("Update FICO / AP status",qid,q.status);
 saveDb();closeModal();render();toast("AP queue status updated");
}

function renderReports(){
```

```
  return pageHead("Reports & Audit","KASE ERP › Placements › Reports",btn("Export all audit records","exportRows('audit')"))+
  `<div class="cards">
    ${metric("Talent records",db.candidates.length,"Candidate register")}
    ${metric("Verified employers",db.employers.filter(e=>e.status==="Verified").length,"Employer register")}
    ${metric("Open vacancies",db.jobs.filter(j=>j.status==="Open").length,"Vacancy register")}
    ${metric("FICO / AP requests",db.fico.length,"Payment workflow")}
  </div>
  <section class="card">
    <div class="card-head"><h2>Data exports</h2></div>
    <div class="actions">
      ${btn("Talent pool CSV","exportRows('candidates')")}
      ${btn("Employers CSV","exportRows('employers')")}
      ${btn("Vacancies CSV","exportRows('jobs')")}
      ${btn("Drives CSV","exportRows('drives')")}
      ${btn("Interviews CSV","exportRows('interviews')")}
      ${btn("Retention CSV","exportRows('retention')")}
      ${btn("FICO / AP CSV","exportRows('fico')")}
    </div>
  </section>
  <section class="card">
    <div class="card-head"><h2>Audit trail</h2>${btn("Refresh","render()")}</div>
    ${table(["Date / time","User","Action","Record","Details"],db.audit.slice(0,100),a=>[
      esc(a.date),esc(a.user),esc(a.action),esc(a.entity),esc(a.details)
    ])}
  </section>
  <p class="muted">This standalone prototype saves demonstration data in this browser using localStorage. A production implementation requires server-side persistence, authentication and authorisation, document storage, website integration, and live eOffice/FICO interfaces.</p>`;
}
```

```
function exportRows(type){
 let rows=[];
 if(type==="retention"){
  rows=db.candidates.filter(c=>c.status==="Retention Monitoring"||c.status==="Placed").map(c=>({
   candidateId:c.id,name:c.name,employer:employerName(c.employerId),
   placedAt:c.placedAt,milestone30:milestoneStatus(c,30),
   milestone60:milestoneStatus(c,60),milestone90:milestoneStatus(c,90)
  }));
 }else if(type==="verification"){
  rows=db.candidates.filter(c=>c.status==="Awaiting Verification"||c.proofStatus!=="Not Submitted");
 }else{
  rows=db[type]||[];
 }
 if(!rows.length){toast("No records to export");return;}

 const flat=rows.map(r=>{
  const o={};
  Object.entries(r).forEach(([k,v])=>{
   if(Array.isArray(v)||typeof v==="object"&&v!==null)o[k]=JSON.stringify(v);
   else o[k]=v;
  });
  return o;
 });
 const keys=[...new Set(flat.flatMap(o=>Object.keys(o)))];
 const csv="\uFEFF"+[keys,...flat.map(o=>keys.map(k=>o[k]??""))]
  .map(row=>row.map(v=>'"'+String(v??"").replace(/"/g,'""')+'"').join(","))
  .join("\r\n");
 const url=URL.createObjectURL(new Blob([csv],{type:"text/csv;charset=utf-8"}));
 const a=document.createElement("a");
 a.href=url;
```

```
 a.download="KASE_Placements_"+type+".csv";
 a.click();
 URL.revokeObjectURL(url);
 logAction("Export CSV",type,rows.length+" rows");
 toast("CSV export started");
}

function exportDashboard(){exportRows("candidates");}

$("globalSearch").addEventListener("keydown",event=>{
 if(event.key!=="Enter")return;
 if(activeModule==="talent"||activeModule==="employers"||activeModule==="vacancies"){
  render();
 }else{
  const q=event.target.value.toLowerCase();
  const c=db.candidates.find(x=>JSON.stringify(x).toLowerCase().includes(q));
  if(c){
   openModule("talent");
   setTimeout(()=>viewCandidate(c.id),0);
  }else{
   toast("No matching candidate found in the current search");
  }
 }
});

$("globalSearch").addEventListener("input",()=>{

if(activeModule==="talent"||activeModule==="employers"||activeModule==="vacancies")render();
});

$("dateLabel").textContent=new Date().toLocaleDateString("en-GB",{day:"2-
digit",month:"short",year:"numeric"});
```

```
$("logoutButton").style.display="none";
render();
</script>
</body>
</html>
```